

GoMuseum开发指南 - 项目初始化

Step 0 - 项目初始化

概览

目标: 建立完整的GoMuseum项目基础架构，包括前后端目录结构、配置文件、开发环境和CI/CD管道。

Agent角色:

- **架构师:** 负责整体项目结构设计和技术选型
- **DevOps工程师:** 负责CI/CD配置和环境搭建
- **前端架构师:** 负责React/Next.js项目结构设计
- **后端架构师:** 负责Django REST API架构设计

工作量估算:

- 时间投入: 4-6小时
- 复杂度: 中等
- 依赖项: 环境准备完成（参考前置章节）

软件安装清单与指导

必需软件清单

- ☑ Node.js 20.11.0 + npm 10.4.0
- ☑ Python 3.11.7 + pip
- ☑ Docker 25.0.2 + Docker Compose 2.24.5
- ☑ PostgreSQL 16.1
- ☑ Redis 7.2.4
- ☑ Git 2.43.0

开发工具推荐

- ☑ VS Code + 推荐插件包
- ☑ Postman/Insomnia (API测试)
- ☑ pgAdmin/DBeaver (数据库管理)
- ☑ Redis Commander (Redis管理)

环境验证脚本

```
bash
```

```
#!/bin/bash
```

```
# verify-environment.sh - 一键环境验证
```

```
echo "🔍 GoMuseum环境验证开始..."
```

```
# 创建验证报告
```

```
REPORT_FILE="environment-report.txt"
```

```
echo "GoMuseum环境验证报告 - $(date)" > $REPORT_FILE
```

```
verify_version() {
```

```
    local cmd=$1
```

```
    local expected=$2
```

```
    local actual=$(($cmd 2>/dev/null || echo "未安装"))
```

```
    if [[ $actual == *"$expected"* ]]; then
```

```
        echo "✅ $cmd: $actual" | tee -a $REPORT_FILE
```

```
    else
```

```
        echo "❌ $cmd: 期望 $expected, 实际 $actual" | tee -a $REPORT_FILE
```

```
        return 1
```

```
    fi
```

```
}
```

```
# 验证各项依赖
```

```
verify_version "node --version" "v20.11"
```

```
verify_version "python3 --version" "Python 3.11"
```

```
verify_version "docker --version" "Docker version 25"
```

```
verify_version "psql --version" "psql (PostgreSQL) 16"
```

```
verify_version "redis-server --version" "Redis server v=7.2"
```

```
echo "📄 验证报告已生成: $REPORT_FILE"
```

```
echo "🎉 环境验证完成! "
```

Claude Code完整指令

请首先阅读并理解以下架构文档章节：

- 系统架构概述

- 技术栈选择
- 数据库设计
- API设计规范
- 前端架构设计
- 部署架构

基于这些架构文档，执行以下项目初始化任务。

项目初始化 - GoMuseum

你现在需要调用相关agent角色来完成GoMuseum项目的初始化，如果没有找到可调用agent，请你扮演此角色：

🏗️ 架构师角色

请作为架构师，基于博物馆数字化平台的需求：

1. 设计整体项目架构（前后端分离、微服务考虑）
2. 定义核心模块结构（用户系统、展品管理、互动体验等）
3. 设计数据流和API交互模式
4. 制定技术架构决策文档

🛠️ DevOps工程师角色

请作为DevOps工程师：

1. 创建Docker开发环境配置
2. 设置GitHub Actions CI/CD流程
3. 配置三环境部署策略
4. 建立监控和日志系统基础

🎨 前端架构师角色

请作为前端架构师：

1. 创建Flutter项目结构（Clean Architecture + Feature-first）
2. 配置依赖注入、状态管理、网络层
3. 设计组件库架构（原子设计模式）
4. 配置路由、主题、多语言支持
5. 集成相机识别和支付功能

🚧 后端架构师角色

请作为后端架构师：

- 1. 创建FastAPI项目结构
- 2. 设计REST API架构
- 3. 配置PostgreSQL数据模型和Alembic迁移
- 4. 设置Redis缓存策略
- 5. 配置异步任务队列

📁 具体执行任务

基于以上角色分析，请按照以下目录结构创建项目：

目标目录结构

```
gomuseum/  
├── backend/ (FastAPI + SQLAlchemy + Alembic)  
├── frontend/gomuseum_app/ (Flutter Clean Architecture)  
├── deployment/ (Docker + K8s + Terraform)  
├── docs/ (API + Architecture + User docs)  
├── monitoring/ (Prometheus + Grafana)  
├── scripts/ (自动化脚本)  
└── tools/ (开发工具)
```

阶段1: 项目结构创建

创建完整的项目目录结构，严格按照Clean Architecture原则：

- backend/ 使用FastAPI + SQLAlchemy架构
- frontend/ 使用Flutter Clean Architecture（data/domain/presentation分层）
- deployment/ 包含多环境Docker配置
- monitoring/ 配置监控和日志系统

阶段2: 核心配置文件

生成所有必要的配置文件：

- pubspec.yaml (Flutter依赖)
- requirements/ (Python依赖分环境管理)
- docker-compose.yml (多环境配置)
- .env.example (环境变量模板)
- pyproject.toml, analysis_options.yaml等配置文件

阶段3: 基础架构代码

创建项目基础架构：

- Flutter的依赖注入、网络层、存储层

- FastAPI的路由、中间件、数据库连接
- Alembic数据库迁移配置
- API端点基础结构
- 认证和权限框架

阶段4: 开发工具配置

设置开发效率工具：

- Pre-commit hooks
- 测试框架配置 (pytest + flutter_test)
- 代码格式化配置
- Makefile和自动化脚本
- IDE配置文件

请严格按照上述目录结构执行，每完成一个阶段就报告进度和输出文件清单。

2) 项目目录结构生成

期望的完整目录结构:

gomuseum/

|—— README.md

|—— docker-compose.yml

|—— docker-compose.staging.yml

|—— docker-compose.production.yml

|—— .env.example

|—— .gitignore

|—— Makefile

|—— .github/

| |—— workflows/

|—— backend/

| |—— app/

| | |—— models/

| | |—— schemas/

| | |—— api/

| | | |—— v1/

| | |—— services/

| | |—— core/

| | |—— utils/

| | |—— workers/

| |—— tests/

| | |—— unit/

| | |—— integration/

| | |—— e2e/

| |—— alembic/

| | |—— versions/

| |—— scripts/

| |—— requirements/

| |—— Dockerfile

| |—— Dockerfile.prod

| |—— .env.example

| |—— pyproject.toml

| |—— .flake8

| |—— .coveragerc

|—— frontend/

| |—— gomuseum_app/

| |—— lib/

```
| | |—— config/
| | |—— core/
| | |—— di/
| | |—— error/
| | |—— network/
| | |—— storage/
| | |—— utils/
| | |—— features/
| | |—— auth/
| | |—— data/
| | |—— datasources/
| | |—— models/
| | |—— repositories/
| | |—— domain/
| | |—— entities/
| | |—— repositories/
| | |—— usecases/
| | |—— presentation/
| | |—— providers/
| | |—— pages/
| | |—— widgets/
| | |—— recognition/
| | |—— data/
| | |—— domain/
| | |—— presentation/
| | |—— chat/
| | |—— data/
| | |—— domain/
| | |—— presentation/
| | |—— payment/
| | |—— data/
| | |—— domain/
| | |—— presentation/
| | |—— profile/
| | |—— data/
| | |—— domain/
| | |—— presentation/
```

```
| | └── shared/
| |   ├── widgets/
| |   ├── models/
| |   └── utils/
| ├── test/
| | ├── features/
| | ├── shared/
| | └── core/
| ├── integration_test/
| ├── assets/
| | ├── images/
| | | └── icons/
| | ├── fonts/
| | └── data/
| ├── android/
| ├── ios/
| ├── web/
| ├── pubspec.yaml
| ├── analysis_options.yaml
| ├── Dockerfile
| └── README.md
└── deployment/
    ├── nginx/
    | ├── sites-available/
    | └── ssl/
    ├── docker/
    | ├── postgres/
    | ├── redis/
    | └── monitoring/
    |   └── grafana/
    ├── kubernetes/
    ├── ansible/
    | ├── inventory/
    | ├── playbooks/
    | └── roles/
    ├── terraform/
    └── modules/
```

```
| └── scripts/
| └── docs/
| └── requirements/
| └── api/
| └── architecture/
| └── deployment/
| └── development/
| └── user/
| └── monitoring/
| └── logs/
| └── metrics/
| | └── prometheus/
| | └── grafana/
| └── alerts/
| └── rules/
| └── scripts/
| └── tools/
| └── generators/
| └── validators/
| └── formatters/
└── .vscode/
```

总结

输出文件清单

项目初始化完成后，应包含以下关键文件：

配置文件 (12个):

- package.json (根目录)
- frontend/package.json
- backend/requirements.txt
- docker-compose.yml
- .env.example
- .gitignore

- .eslintrc.js
- prettier.config.js
- tsconfig.json
- tailwind.config.js
- next.config.js
- backend/config/settings.py

基础架构文件 (15个):

- 前端布局组件和路由配置
- 后端应用模块结构
- 数据库模型定义
- API序列化器和视图
- 认证中间件
- 错误处理组件

开发工具文件 (8个):

- .husky/pre-commit
- pytest.ini
- jest.config.js
- sonar-project.properties
- .github/workflows/ci.yml
- .github/workflows/deploy.yml
- scripts/setup.sh
- scripts/verify-environment.sh

测试用例及代码覆盖率目标

- 前端单元测试: 组件渲染、Hook逻辑、工具函数

- **后端单元测试:** 模型验证、API端点、业务逻辑
- **集成测试:** API交互、数据库操作、认证流程
- **E2E测试:** 关键用户流程测试

覆盖率目标:

- 单元测试覆盖率: $\geq 80\%$
- 集成测试覆盖率: $\geq 70\%$
- 总体代码覆盖率: $\geq 75\%$

验收

自动化验收脚本

```
bash
```

```
#!/bin/bash
```

```
# acceptance-test.sh - 项目初始化合验测试
```

```
echo "🔧 项目初始化合验测试开始..."
```

```
# 检查目录结构
```

```
echo "📁 检查目录结构..."
```

```
required_dirs=("frontend" "backend" "shared" "docker" "docs" "scripts" ".github")
```

```
for dir in "${required_dirs[@]}; do
```

```
  if [ -d "$dir" ]; then
```

```
    echo "✅ $dir 目录存在"
```

```
  else
```

```
    echo "❌ $dir 目录缺失"
```

```
    exit 1
```

```
  fi
```

```
done
```

```
# 检查配置文件
```

```
echo "⚙️ 检查配置文件..."
```

```
required_files=("package.json" "docker-compose.yml" ".env.example" ".gitignore")
```

```
for file in "${required_files[@]}; do
```

```
  if [ -f "$file" ]; then
```

```
    echo "✅ $file 文件存在"
```

```
  else
```

```
    echo "❌ $file 文件缺失"
```

```
    exit 1
```

```
  fi
```

```
done
```

```
# 检查依赖安装
```

```
echo "📦 检查依赖安装..."
```

```
cd frontend && npm list --depth=0 > /dev/null 2>&1
```

```
if [ $? -eq 0 ]; then
```

```
  echo "✅ 前端依赖安装成功"
```

```
else
```

```
  echo "❌ 前端依赖安装失败"
```

```
  exit 1
```

fi

```
cd ../backend && pip list > /dev/null 2>&1
```

```
if [ $? -eq 0 ]; then
```

```
    echo "✅ 后端依赖安装成功"
```

```
else
```

```
    echo "❌ 后端依赖安装失败"
```

```
    exit 1
```

```
fi
```

```
cd ..
```

检查服务启动

```
echo "🚀 检查服务启动..."
```

```
docker-compose up -d
```

```
sleep 10
```

检查前端服务

```
curl -f http://localhost:3000 > /dev/null 2>&1
```

```
if [ $? -eq 0 ]; then
```

```
    echo "✅ 前端服务启动成功"
```

```
else
```

```
    echo "❌ 前端服务启动失败"
```

```
fi
```

检查后端API

```
curl -f http://localhost:8000/api/health/ > /dev/null 2>&1
```

```
if [ $? -eq 0 ]; then
```

```
    echo "✅ 后端API服务启动成功"
```

```
else
```

```
    echo "❌ 后端API服务启动失败"
```

```
fi
```

检查数据库连接

```
docker-compose exec backend python manage.py check --deploy
```

```
if [ $? -eq 0 ]; then
```

```
    echo "✅ 数据库连接正常"
```

```
else
```

```
echo "❌ 数据库连接失败"
fi

docker-compose down

echo "🎉 项目初始化验收测试完成! "
```

手工验收检查清单

- ☐ 项目目录结构完整
- ☐ 所有配置文件已生成
- ☐ 前端开发服务器可启动 (npm run dev)
- ☐ 后端API服务器可启动 (python manage.py runserver)
- ☐ 数据库迁移成功执行
- ☐ Redis连接正常
- ☐ Docker容器化环境可启动
- ☐ 基础测试套件可运行
- ☐ 代码格式化工具工作正常
- ☐ Git hooks已配置并生效
- ☐ CI/CD流程配置正确
- ☐ 环境变量模板完整

版本管理和CI/CD

初始化后的Git workflow

1. 创建初始提交:

```
bash
```

```
git add .  
git commit -m "feat: initial project setup"
```

- Add frontend Next.js structure
- Add backend Django structure
- Configure Docker development environment
- Set up CI/CD workflows
- Add project documentation"

2. 创建开发分支:

```
bash
```

```
git checkout -b feature/init/basic-structure  
git push -u origin feature/init/basic-structure
```

3. 设置分支保护:

- 在GitHub上配置main分支保护规则
- 启用CI检查要求
- 配置自动部署到staging环境

CI/CD Pipeline激活

- 触发条件: Push到feature/init/*分支
- 执行步骤:
 1. 代码质量检查 (ESLint, Black)
 2. 单元测试运行
 3. 构建验证
 4. Docker镜像构建
 5. 安全性扫描
 6. 部署到staging环境 (可选)

下一步计划

项目初始化完成后，开发流程将进入MVP功能开发阶段。

下一步: 项目初始化完成并通过验收测试后，可以进入《GoMuseum开发指南-MVP核心功能.md》阶段。